

Lecture 1: Introduction

COMP2221: Functional programming

Maximilien Gadouleau

`m.r.gadouleau@durham.ac.uk`

Course organisation

Course organisation: methods

Lectures

- 10 lectures overall
- 2 per week

Practicals

- Programming requires practice
- Tackle exercises yourself (before the practical session) first
- Your chance to ask questions and get feedback
- First practicals start on week beginning Monday, the 19th of January

Office hours

- Tuesdays 11-12 in my office (MCS 2022)
- Could schedule alternative times by email

- Course mainly follows Graham Hutton's *Programming in Haskell* (2016)
- Available at the library
- Slides for the first 10 chapters are available at <http://www.cs.nott.ac.uk/~pszgmh/pih.html>

- By exam
- *Knowledge and comprehension*: How do things work in Haskell? Why do they work?
- *Application*: What does some code do? Can you write code to solve problem X?
- *Evaluation*: What are the underlying concepts? What properties does a solution have?
- Past papers available and sample paper will be provided

Functional programming and Haskell

What is functional programming?

- A programming *paradigm* where the building block of computation is the *application of functions* to arguments.
- Functional programs specify a data-flow to describe *what* computations should proceed
- Algebraic programming style dominated by function application and composition

In contrast, what is imperative programming?

- A programming *paradigm* where the building block of computation is the *modification of stored values*.
- Imperative programs specify a control-flow to describe *how* computations should proceed
- Algorithmic programming style dominated by variable assignments, loops, conditional statements

Effectful vs. pure functions

```
y1 = f(1)  
y2 = f(1)
```

Will `y1 == y2`? How could it not?

Effectful vs. pure functions

```
y1 = f(1)
y2 = f(1)
```

Will `y1 == y2`? How could it not?

If `f` has some internal state that affects the answer:

```
state = 0
def f(n):
    global state
    state += 1
    return n + state

print(f(1)) => "2"
print(f(1)) => "3"
```

Definition (Side effect)

A side effect is a (hidden) state change that results from a function modifying or relying upon objects external to its parameter list.

Definition (Pure function)

A pure function is a function that takes all its inputs as arguments and produces all its outputs as results.

⇒ A pure function is a function without side effects

What is a functional programming language?

A functional programming language:

- *Supports* and *encourages* a programming style with function application and composition as basic building blocks
- Forbids variable assignment and side effects $\Rightarrow$ “Pure functional”
- Makes *reasoning* about code simpler (for humans and compilers)

Why are C/Java/Python not functional programming languages?

- ✓ It is indeed *possible* to write in a functional style in these languages. . .
- ✗ but they do not enforce it.
- ✗ Moreover, the language-level support is weak.
- ✓ In contrast, Haskell is a purely functional (side effect free) language, and built from scratch for functional programming.

Example: computing $n!$

Imperative style

```
factorial = 1
for i in range(1, n+1):
    factorial = factorial * i
```

Functional style

$$F_n = \begin{cases} 1 & n = 1 \\ nF_{n-1} & \text{otherwise} \end{cases}$$

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)
```

Which implementation maps more naturally onto a computer?

Which implementation is more convenient for the programmer?

Functional style

$$F_n = \begin{cases} 1 & n = 1 \\ nF_{n-1} & \text{otherwise} \end{cases}$$

-- Compute the factorial of an integer

```
fac :: Int -> Int
```

```
fac 1 = 1
```

```
fac n = n * fac (n - 1)
```

- Semantics of complex code fragments is given implicitly: reader has to reconstruct it
- We can think about how to write it for humans to understand
- Comments (or literate programming) can help

Development environment

- GHC (Glasgow Haskell Compiler) can be used as an interpreter `ghci` and compiler `ghc`
- Available freely from www.haskell.org/download
- `ghcup` is the main installer for the general purpose language Haskell
- De-facto standard implementation

Standard library

- Ease of use of languages often determined by standard library
- Haskell has a large standard library, and is particularly strong manipulating lists
- In practicals, you will redo some of its functionalities for practice purposes

Introduction to types

Definition: What are types?

Definition (Type)

A type is a *collection* of related values.

A type can be described by specifying

- the set of data elements it covers and
- the operations it supports.

Example

- **Bool** the two logical values **True** and **False**.
- **Integer** $\rightarrow$ **Integer** the set of all functions that take an **Integer** as input and produce an **Integer** as output.

Motivation: Why do we need types?

Example in C/Java

```
int a = 4;
```

```
int b = 3;
```

```
double c = a/b;
```

```
double a = 4;
```

```
double b = 3;
```

```
double c = a/b;
```

Motivation: Why do we need types?

Example in C/Java

```
int a = 4;
```

```
int b = 3;
```

```
double c = a/b;
```

```
double a = 4;
```

```
double b = 3;
```

```
double c = a/b;
```

Result depends on input types.

Since computers represent *everything* as sequences of bits, types are also required to interpret these bit patterns.

Motivation: Why do we need types?

- Mathematics and programming rely on the notion of *types*
- Tell us *how* to interpret a variable
- Provide restrictions on valid *operations*
- Are required to know what a bit pattern means

Implementation

Find the correct implementation of an operator.

Correctness

Check whether an operation on some data is valid and/or well-defined.

Check whether a code fragment is correct (type safety).

Documentation

Document the code's semantics for the reader.

Type System Classification

- Strongly vs. weakly typed languages
- Statically vs. dynamically typed languages

Static type checking

Translators must check for *type correctness*

Definition (Statically typed language)

Type safety is checked at translation time.

⇒ invalid types result in translation error

```
-- Invalid
```

```
foo :: a -> Int
```

```
foo f = 1 + f
```

Definition (Dynamically typed language)

Type safety is checked at run time.

⇒ invalid types only detected as soon as used

```
# Fine as long as f supports addition with a number  
def foo(f):  
    return 1 + f
```

- How does the translator determine the type of an expression?

Explicit annotation

Programmer annotates all variables with type information (e.g. C/Java)

Type inference

Translator *infers* the types of variables based on the operations used (e.g. Haskell)

Duck typing

Translator/runtime just tries the operation, if it succeeds, that was a valid type! (Python)

Haskell is

Strongly, statically typed.

⇒ every well-formed expression has exactly one type, these types are known at *compile time*

Notation and inspection

Attaching types

Haskell's notation for “e is of type T” is spelt

```
e :: T
-- False is of type Bool
False :: Bool
-- not is of type Bool -> Bool
not :: Bool -> Bool
```

What type does X have?

Every valid expression in Haskell must have a valid type.

You can ask GHCi what the type of an expression is with the command
`:type expr`

```
Prelude> :type sum
sum :: Num a => [a] -> a
```

Functions have types

- Functions have types in all programming languages, Haskell makes this particularly explicit

Functions of one argument “unary”

Map from one type to another

```
not :: Bool -> Bool
```

```
and :: [Bool] -> Bool
```

Functions of two arguments “binary”

Map from two types to another

```
add :: (Int, Int) -> Int
```

“add eats two Ints and returns an Int”

- (Almost) all functions in Haskell are written in *curried* form

```
-- "uncurried"
```

```
add' :: (Int, Int) -> Int
```

```
add' (x, y) = x + y
```

```
-- "curried"
```

```
add'' :: Int -> Int -> Int
```

```
add'' x y = x + y
```

Currying conventions

⇒ To avoid messy syntax, this leads to associativity rules for `->` and function application.

`->` associates to the right

```
Int -> Int -> Int -> Int
-- Means
Int -> (Int -> (Int -> Int))
```

Function application associates to the left

```
mult x y z
-- Means
((mult x) y) z
```